Claves y restricciones en SQL

Del modelo E-R al esquema relacional en DDL: claves primarias, restricciones y claves foráneas

Semana 06 - Sesión 12

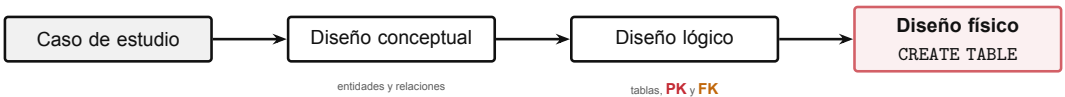
Rel Guzman



Logro de la sesión

Al finalizar, podrás definir **claves primarias** y **restricciones**, crear una base de datos y conectar sus tablas mediante **claves foráneas**.

Dónde estamos



DDL

DDL es el **lenguaje de definición de datos**: permite crear y modificar la estructura de la base de datos. `CREATE TABLE` crea sus tablas.

Claves y restricciones

Paso 4: ¿quién es cada fila?

```
CREATE TABLE cliente
(
   id       INT          PRIMARY KEY,
   nombre   VARCHAR(80),
   correo   VARCHAR(120)
);
```



la **PK** del dibujo
es esta línea

Sin el `id` hay un problema: si dos clientes se llaman igual, **no se pueden distinguir**. La **clave primaria** es la columna que identifica a una sola fila; no se repite y nunca queda vacía.

Y con eso la instrucción está completa: **eso es todo un** `CREATE TABLE`.

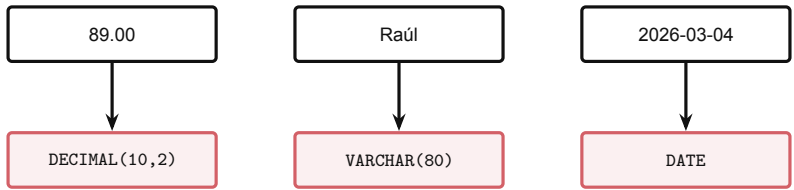
La instrucción completa, con sus nombres

```
CREATE TABLE cliente (
   id       INT            PRIMARY KEY,
   nombre   VARCHAR(80)    NOT NULL,
   correo   VARCHAR(120)   UNIQUE
);
```

Parte	Cómo se llama
`cliente`	el **nombre de la tabla**
`nombre`	el **nombre de la columna**
`VARCHAR(80)`	el **tipo de dato**
`PRIMARY KEY`	la **clave primaria**
`NOT NULL, UNIQUE`	las **restricciones**

Faltan las dos últimas, y son las que siguen.

El tipo dice qué clase de cosa se guarda



Tipo	Para qué	Ejemplo
`INT`	Números enteros	`id, cantidad`
`VARCHAR(n)`	Texto, hasta `n` caracteres	`nombre VARCHAR(80)`
`DECIMAL(p,d)`	Dinero	`precio DECIMAL(10,2)`
`DATE`	Fecha	`fecha DATE`
`BOOLEAN`	Verdadero o falso	`activo BOOLEAN`

Práctica 2 Elige el tipo de cada columna

Dato que hay que guardar	Tipo	¿Por qué?
El precio de un producto		
El DNI del cliente		
La fecha del pedido		
Cuántas unidades se pidieron		
Si el producto sigue a la venta		

Pista: pregúntate dos cosas. ¿Se harán **cuentas** con ese dato? ¿Puede empezar con **cero**?

Las restricciones: reglas que toda fila cumple

Restricción	Qué obliga	En la tienda
`NOT NULL`	No puede quedar vacía	Un cliente sin nombre no sirve
`UNIQUE`	No se repite en dos filas	Dos clientes no comparten correo
`AUTO_INCREMENT`	El gestor numera solo	`id` 1, 2, 3…sin pensarlo
`DEFAULT` *`valor`*	Si no se indica, usa ese	`activo BOOLEAN DEFAULT TRUE`

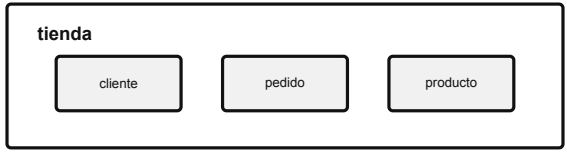
Se escriben **después del tipo**, en la misma línea. Son opcionales: una columna con nombre y tipo ya funciona.

`PRIMARY KEY` ya incluye `NOT NULL` y `UNIQUE`; por eso casi siempre se escribe junto con `AUTO_INCREMENT` y nada más.

Preguntas



Antes de las tablas: crear la base y entrar en ella



la base de datos es la **caja**; las tablas van dentro

```
CREATE DATABASE tienda;
USE tienda;

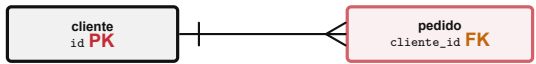
CREATE TABLE cliente ( …);
```

`USE` dice en cuál se trabaja: desde esa línea, todo lo que se escriba se crea ahí dentro. Si se olvida, el gestor responde `No database selected`.

En palabras simples

Es el «abrir carpeta» antes de guardar. Si no abres ninguna, el archivo aparece en cualquier parte.

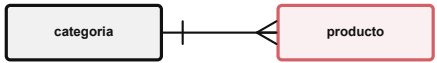
`FOREIGN KEY`: la conexión, por fin escrita



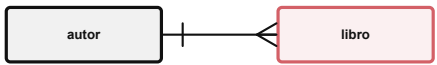

```sql
CREATE TABLE pedido (
   id           INT    PRIMARY KEY AUTO_INCREMENT,
   fecha        DATE   NOT NULL,
   cliente_id   INT    NOT NULL,
   FOREIGN KEY (cliente_id) REFERENCES cliente(id)
);
```

Son dos líneas, no una: arriba se declara `cliente_id` como cualquier columna; abajo se dice **a dónde apunta**. Esa segunda línea conecta cada pedido con un cliente existente.

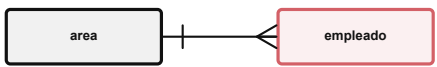
Práctica 3 ¿En qué tabla va la clave foránea?



la columna _____ va en la tabla _____



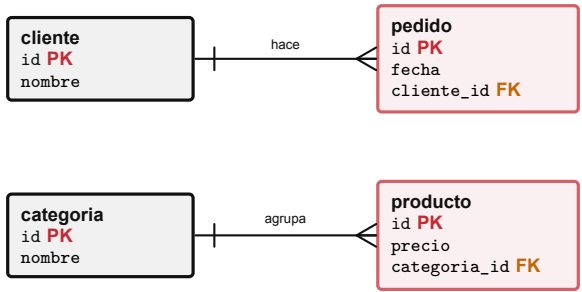
la columna _____ va en la tabla _____



la columna _____ va en la tabla _____

Pista: la clave foránea siempre va del lado de la **pata de gallo**, y se llama como la otra tabla más `_id`.

La tienda de hoy: cuatro tablas, dos conexiones



Una tabla no puede apuntar a otra que todavía no existe

Por eso el orden importa: primero las que **no apuntan a nadie**. **Numera del 1 al 4**:

`producto` _____ `cliente` _____ `pedido` _____ `categoria` _____

La tienda completa, de principio a fin en MySQL

```sql
CREATE DATABASE tienda;   USE tienda;
```

```sql
CREATE TABLE cliente (
   id INT PRIMARY KEY AUTO_INCREMENT,
   nombre VARCHAR(80) NOT NULL,
   correo VARCHAR(120) UNIQUE
);

CREATE TABLE categoria (
   id INT PRIMARY KEY AUTO_INCREMENT,
   nombre VARCHAR(80) NOT NULL
);
```

```sql
CREATE TABLE pedido (
   id INT PRIMARY KEY AUTO_INCREMENT,
   fecha DATE NOT NULL,
   cliente_id INT NOT NULL,
   FOREIGN KEY (cliente_id)
      REFERENCES cliente(id)
);

CREATE TABLE producto (
   id INT PRIMARY KEY AUTO_INCREMENT,
   precio DECIMAL(10,2) NOT NULL,
   categoria_id INT NOT NULL,
   FOREIGN KEY (categoria_id)
      REFERENCES categoria(id)
);
```

Todo lo de hoy cabe aquí: una base, cuatro tablas, sus tipos, sus claves primarias y sus dos claves foráneas.

Práctica 4 Ejecución del script en MySQL

Copia el código completo de la diapositiva anterior en una nueva pestaña de MySQL Workbench y ejecútalo **de arriba abajo**.

```
SHOW TABLES;

DESCRIBE cliente;
DESCRIBE pedido;
```

Comprueba el resultado

Deben aparecer las tablas `cliente`, `categoria`, `pedido` y `producto`. Revisa que `id` sea la clave primaria y que `cliente_id` aparezca en `pedido`.

Si aparece un error, identifica la instrucción que falló, corrígela y vuelve a ejecutar esa instrucción.

La misma tienda, en SQL Server

```
CREATE DATABASE tienda;   GO   USE tienda;   GO
```

```
CREATE TABLE cliente (
   id INT IDENTITY(1,1) PRIMARY KEY,
   nombre NVARCHAR(80) NOT NULL,
   correo NVARCHAR(120) UNIQUE
);

CREATE TABLE categoria (
   id INT IDENTITY(1,1) PRIMARY KEY,
   nombre NVARCHAR(80) NOT NULL
);
```

```
CREATE TABLE pedido (
   id INT IDENTITY(1,1) PRIMARY KEY,
   fecha DATE NOT NULL,
   cliente_id INT NOT NULL,
   FOREIGN KEY (cliente_id)
      REFERENCES cliente(id)
);

CREATE TABLE producto (
   id INT IDENTITY(1,1) PRIMARY KEY,
   precio DECIMAL(10,2) NOT NULL,
   categoria_id INT NOT NULL,
   FOREIGN KEY (categoria_id)
      REFERENCES categoria(id)
);
```

Lo marcado es lo único que cambia: `IDENTITY(1,1)` en lugar de `AUTO_INCREMENT`, `NVARCHAR` en lugar de `VARCHAR`, y `GO` para separar instrucciones. **Todo lo demás es idéntico.**

La misma tienda, en SQLite

```
-- no hay CREATE DATABASE ni USE: la base de datos es un archivo
```

```
PRAGMA foreign_keys = ON;
```

```
CREATE TABLE cliente (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   nombre TEXT NOT NULL,
   correo TEXT UNIQUE
);

CREATE TABLE categoria (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   nombre TEXT NOT NULL
);
```

```
CREATE TABLE pedido (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   fecha TEXT NOT NULL,
   cliente_id INTEGER NOT NULL,
   FOREIGN KEY (cliente_id)
      REFERENCES cliente(id)
);

CREATE TABLE producto (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   precio REAL NOT NULL,
   categoria_id INTEGER NOT NULL,
   FOREIGN KEY (categoria_id)
      REFERENCES categoria(id)
);
```

SQLite tiene **muy pocos tipos**: `INTEGER`, `TEXT` y `REAL` cubren todo, y las fechas se guardan como texto. **Y las claves foráneas vienen apagadas**: sin el `PRAGMA` se aceptan y no se vigilan.

Cada gestor tiene su propia notación

Lo que hay que llevarse

El **SQL que aprendimos hoy es el mismo en todos**. Lo que cambia son **detalles**: cómo se numera solo el `id`, cómo se llama el tipo de texto, si existe o no `CREATE DATABASE`.

	MySQL	SQL Server	SQLite
Numerar solo el `id`	`AUTO_INCREMENT`	`IDENTITY(1,1)`	`AUTOINCREMENT`
Texto	`VARCHAR(80)`	`NVARCHAR(80)`	`TEXT`
Decimales	`DECIMAL(10,2)`	`DECIMAL(10,2)`	`REAL`
Crear la base	`CREATE DATABASE`	`CREATE DATABASE`	es un archivo

Por eso no hay que aprender tres idiomas: se aprende **uno**, y al cambiar de gestor se consulta la tabla de equivalencias. `CREATE TABLE`, `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL` y `UNIQUE` son iguales en los tres.

Práctica 5 Ahora prueba guardar y consultar

Las tablas ya existen, pero están **vacías**. Escribe estas dos instrucciones y mira qué responde el gestor:

```
-- 1. guardar una fila
INSERT INTO cliente (nombre, correo)
VALUES ('Raúl', 'raul@mail.com');

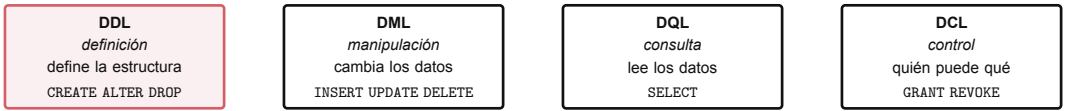
-- 2. verla
SELECT * FROM cliente;
```

`INSERT INTO` dice **en qué tabla y en qué columnas**; `VALUES` trae los datos, en el mismo orden. `SELECT * FROM` es «muéstrame todo de esta tabla».

Dos cosas para comprobar

¿Qué valor le tocó al `id`, si no lo escribimos? Y prueba un pedido con `cliente_id = 97`: **¿qué contesta?**

Lo de hoy fue DDL: ¿y el resto?



Las cuatro son SQL; se agrupan por **para qué sirven**. Hoy solo construimos el **envase**: qué tablas hay y qué columnas tienen. Con el envase hecho, lo que sigue es **llenarlo y consultarlo**.

Cierre: completa en voz alta

- Una columna se escribe siempre con dos cosas: _____

Referencias

Coronel, C., Morris, S., & Rob, P. (2011). *Bases de datos: diseño, implementación y administración* (9.ª ed.). Cengage Learning.

Elmasri, R., & Navathe, S. (2007). *Fundamentos de sistemas de bases de datos* (5.ª ed.). Addison Wesley.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill.



UTP Universidad Tecnológica del Perú